

CSCE 636 DEEP LEARNING · MODULE 02 · LECTURE NOTES

Backpropagation

The chain rule, organised so it runs once.

MODULE OUTCOMES

Derive backpropagation as reverse-mode automatic differentiation.

Explain why reverse mode and not forward mode.

Implement an autodifferentiation engine.

Verify gradients numerically.

Diagnose vanishing and exploding gradients.

1 Reverse-mode automatic differentiation

FORWARD AND REVERSE MODE DIFFER BY WHICH END YOU START FROM

Both compute exact derivatives by systematic application of the chain rule. Neither is finite differencing (which is approximate and unstable) nor symbolic differentiation (which produces expressions that explode in size). **Automatic differentiation is a third thing** and the distinction is worth being clear about.

Forward mode propagates the derivative with respect to one input forward through the graph. One pass gives you the derivative of *every* output with respect to *that one* input, so the cost is one pass per input.

Reverse mode propagates the derivative of one output backward. One backward pass gives the derivative of *that one* output with respect to *every* input, so the cost is one pass per output.

A loss is a single scalar output, and a network has millions of parameters. So reverse mode delivers all the gradients in one backward pass costing roughly the same as the forward pass. **That asymmetry is the entire reason training large models is affordable** — with forward mode, each parameter would need its own pass, and training would be a million times more expensive.

```

class Value:
    def __init__(self, data, parents=(), op=""):
        self.data, self.grad = data, 0.0
        self._parents, self._backward = parents, lambda: None

    def __mul__(self, other):
        out = Value(self.data * other.data, (self, other), "*")
        def _backward():
            # ACCUMULATE, do not assign: a value used twice receives
            # gradient contributions from BOTH consumers.
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad
        out._backward = _backward
        return out

def backward(root):
    topo, seen = [], set()
    def build(v):
        if v in seen: return
        seen.add(v)
        for p in v._parents: build(p)
        topo.append(v)
    build(root)
    root.grad = 1.0
    for v in reversed(topo):      # every consumer before its producer
        v._backward()

```

Two ideas carry the whole implementation: accumulate rather than assign, and process in reverse topological order. The += is the bug everyone writes as = the first time — and it only shows up when a value feeds more than one consumer, which in a simple test it may not. **Everything else is adding more operators.**

IT IS A COMPUTATIONAL GRAPH, NOT A NETWORK

The engine above knows nothing about layers, neurons, activations, or networks. It knows operations and their local derivatives, and how to compose them.

So anything differentiable can be trained by it — a renderer (CSCE 647), a physics simulator (CSCE 649), a relaxed sorting operation, a protein structure predictor, an optimiser's own hyperparameters.

Which is why the major frameworks are differentiation libraries with neural network utilities attached, rather than neural network libraries — a distinction visible in their API design, and one that becomes important the moment you want to differentiate through something that is not a standard layer.

And it is why 'differentiable X' became a research programme. Differentiable rendering, differentiable physics, differentiable sorting, differentiable programming — **once the gradient is free, the only question is whether X can be written differentiably**, which turns the design problem into finding smooth relaxations of discrete operations. Module 11 is an instance of exactly this.

2 Verifying it

```
def grad_check(f, x, eps=1e-5):
    analytic = backward_gradient_of(f, x)
    for i in random_subset(range(len(x))):
        xp, xm = x.copy(), x.copy()
        xp[i] += eps; xm[i] -= eps
        numeric = (f(xp) - f(xm)) / (2*eps)      # CENTRAL difference
        rel = abs(numeric - analytic[i]) / max(1e-8,
            abs(numeric) + abs(analytic[i]))
    assert rel < 1e-5
```

CAVEATS, each of which causes a FALSE failure:

- * use DOUBLE precision -- in float32 the subtraction cancels
- * use RELATIVE error -- gradients span many orders of magnitude
- * check a small RANDOM SUBSET -- checking all is too slow
- * avoid non-differentiable points -- ReLU at exactly 0 will legitimately fail

A false failure on correct code is worse than no check at all, because it sends you looking for a bug that is not there. **The float32 cancellation is the one that wastes the most time:** the central difference subtracts two nearly equal numbers, which is exactly CSCE 620 Module 01's catastrophic cancellation, and in single precision the result can have no correct digits. **Run gradient checks in double.**

3 Vanishing and exploding gradients

$$\partial L / \partial w_1 = \partial L / \partial a_n \cdot \partial a_n / \partial a_{n-1} \cdot \dots \cdot \partial a_1 / \partial w_1$$

The gradient reaching an early layer is a product of n Jacobians. If each factor has magnitude consistently below one, the product decays exponentially in depth — **vanishing** — and the early layers receive essentially no learning signal. If each factor exceeds one, the product grows exponentially — **exploding** — and a single update overshoots catastrophically, usually producing NaN. **The sigmoid's derivative peaks at 0.25**, so ten sigmoid layers attenuate the gradient by a factor of roughly a million even in the best case. **That arithmetic is why depth did not work for twenty-five years**, and it is not subtle once stated.

Fix	Addresses	How
ReLU	Vanishing.	Its derivative is exactly 1 wherever the unit is active, so the product does not attenuate through active paths. A one-line change with an enormous effect (Module 01 §3).
He / Xavier initialisation	Both.	Scales the initial weights so that activation variance is preserved from layer to layer — derived from the fan-in and fan-out, not tuned. Before this, deep networks frequently could not be trained at all.
Residual connections	Vanishing.	An identity shortcut gives the gradient a path with factor exactly 1, so it reaches early layers regardless of what the intervening blocks do. See below.
Normalisation layers	Both.	Rescale activations at each layer so neither the forward signal nor the backward gradient drifts in magnitude (Module 04).
Gradient clipping	Exploding.	Caps the gradient norm before the update. Crude, effective, and essentially mandatory for recurrent networks (Module 06).
Gating (LSTM, GRU)	Vanishing.	The cell state provides an additive path through time — the same mechanism as a residual connection , discovered earlier and in a different setting.

Residual connections are the single most important architectural idea in this module. By making each block compute $x + f(x)$ rather than $f(x)$, the gradient flows backward through the identity term unattenuated — **and that is why networks went from about twenty trainable layers to over a thousand**. It is also why the same pattern appears in transformers (Module 07), in U-Nets, and in diffusion models: **the trick is not about images, it is about gradient paths**.

4 Diagnosing it

- **Log the gradient norm per layer, every few hundred steps. A histogram of gradient magnitude across layers is the single best diagnostic available for this class of problem**, and it costs almost nothing.
- **Vanishing looks like:** early layers' gradient norms orders of magnitude smaller than later ones, and a loss that plateaus above chance while the last layer alone continues to improve.

- **Exploding looks like:** a sudden spike in the loss followed by NaN, or a loss that oscillates violently without settling. **The spike is usually one bad batch**, and clipping absorbs it.
- **Dead ReLUs:** units whose output is zero for every input in the batch, which receive no gradient and never recover. **Log the fraction of dead units per layer**; above roughly 40% indicates the learning rate was too high at some point and the layer is now partly destroyed.
- **And NaN propagates instantly through everything. Find the *first* step at which it appears and inspect the input batch**, rather than the architecture — the cause is usually a specific input (a zero divisor, a log of zero, an extreme outlier) rather than a structural problem.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Karpathy — micrograd, and 'The spelled-out intro to neural networks and backpropagation' (free)	The §1 engine, built live on video in two hours. The single best resource for this module.
Baydin, Pearlmutter, Radul & Siskind — Automatic Differentiation in Machine Learning: a Survey (free)	Forward versus reverse mode done properly , and the distinction from symbolic and numerical differentiation.
He, Zhang, Ren & Sun — Deep Residual Learning (free)	The §3 fix. Read the figure showing that a deeper plain network was <i>worse</i> on training error — which is what motivated it.
CS231n — backpropagation and gradient checking notes (free)	The §2 caveats, each explained with the failure it causes.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement a scalar autodifferentiation engine** with add, multiply, power, and tanh.
- 2 **Write = instead of +=** in the backward pass and construct a graph where it gives the wrong answer.
- 3 Extend the engine to tensors and train a two-layer network with it.
- 4 **Implement gradient checking** and verify your engine.
- 5 **Run the check in float32** and observe the false failures.
- 6 Check the gradient of ReLU at exactly zero and explain the result.
- 7 **Train a 15-layer network with sigmoid activations** and log per-layer gradient norms. Plot them.

- 8 Switch to ReLU and plot again. Then add residual connections and plot a third time.
- 9 **Make a network explode** with a high learning rate, then fix it with clipping.
- 10 **Measure the dead ReLU fraction** per layer and relate it to the learning rate used.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Distinguish forward and reverse mode, and say why reverse wins here.
- 2 What two ideas does an autodiff engine need?
- 3 Why must gradients be accumulated rather than assigned?
- 4 Why can anything differentiable be trained?
- 5 Give four caveats for gradient checking and the failure each causes.
- 6 Explain vanishing and exploding gradients from the chain rule.
- 7 Give six fixes and what each addresses.
- 8 Why are residual connections so important?
- 9 Name four gradient pathologies and how each looks in the logs.